



SAKARYA
UYGULAMALI BİLİMLER
ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ

Yapay Zekanın İlkeleri

Proje Ödevi

Öğretim Görevlisi : Doç. Dr. Ekin EKİNCİ
Arş. Gör. Furkan ATBAN

Hazırlayanlar:

Ad: Rümeysa KOLİP
No: 23010903093

Ad: Elif AYTAN
No: 22010903035

1.Proje Tanıtımı ve Amacı.....	2
1.1. Problem Tanımı.....	2
2. Algoritmaların Açıklamaları.....	3
2.1. Breadth-First Search (BFS).....	3
2.2. Depth-First Search (DFS).....	3
2.3. A* Arama Algoritması.....	3
3. Problem Özelleştirmesi ve Test Senaryoları.....	4
4. Algoritmaların Performans Karşılaştırması.....	4
5. Sonuçlar ve Değerlendirme.....	5
6. Kaynakça.....	5
7.Ekran Çıktıları.....	6
1. başlangıç durumu - BFS Algoritması.....	6
1. başlangıç durumu - DFS Algoritması.....	7
1. başlangıç durumu - A* Algoritması.....	8
2. başlangıç durumu - BFS Algoritması.....	9
2. başlangıç durumu - DFS Algoritması.....	10
2. başlangıç durumu - A* Algoritması.....	11
3. başlangıç durumu - BFS Algoritması.....	12
3. başlangıç durumu - DFS Algoritması.....	13
3. başlangıç durumu - A* Algoritması.....	14

1.Proje Tanıtımı ve Amacı

Bu proje, Yapay Zekanın İlkeleri dersi kapsamında "8-Puzzle" olarak bilinen 3x3 kaydırmalı sayı bulmacasını çözmek amacıyla gerçekleştirilmiştir. Projede, başlangıç durumundan hedef duruma ulaşmak için çeşitli arama algoritmaları kullanılmıştır. Bu algoritmalar; Genişlik Öncelikli Arama (Breadth-First Search - BFS), Derinlik Öncelikli Arama (Depth-First Search - DFS) ve A* algoritmasıdır. Bu algoritmaların her biri, belirli başlangıç durumları üzerinden çalıştırılmış, performansları zaman, maliyet ve derinlik açısından karşılaştırılmıştır.

1.1. Problem Tanımı

8-Puzzle, 3x3'lük bir ızgarada 8 taşın yer değiştirilerek hedef duruma ulaşılması gereken bir bulmacadır. Her taş benzersiz bir sayıya sahiptir ve boş bir alan (0) vardır. Amaç, başlangıç durumundan hareketle, taşları kaydırarak hedef duruma en kısa yoldan ulaşmaktır.

Geçiş Kuralları:

- Yukarı (Up)
- Aşağı (Down)
- Sol (Left)
- Sağ (Right)

Boşluğun (0) konumuna bağlı olarak bu hareketler yapılır. Geçiş sırasında taşlar 0 ile yer değiştirir.

2. Algoritmaların Açıklamaları

2.1. Breadth-First Search (BFS)

BFS algoritması, genişlik öncelikli arama prensibiyle çalışır. Başlangıç durumundan itibaren her seviyedeki tüm komşu durumları keşfederek hedef duruma ulaşmaya çalışır. FIFO (First In First Out) mantığı ile çalışır, yani en erken eklenen durum en önce keşfedilir.

Özellikler:

- En kısa yolu garanti eder.
- Bellek kullanımı yüksektir.
- Zaman karmaşıklığı $O(b^d)$ (b: dal sayısı, d: derinlik)

2.2. Depth-First Search (DFS)

DFS algoritması, derinlik öncelikli arama prensibiyle çalışır. Bir yolu tamamen araştırır, eğer çözüm bulamazsa bir adım geri gider ve başka bir yola geçer. LIFO (Last In First Out) mantığıyla çalışır.

Özellikler:

- Bellek kullanımı düşüktür.
- En kısa yolu garanti etmez.
- Sonsuz döngü riski taşır (döngü kontrolü eklenmelidir).
- Zaman karmaşıklığı $O(b^m)$ (b: dal sayısı, m: maksimum derinlik)

2.3. A* Arama Algoritması

A* algoritması, maliyet ($g(n)$) ve hedefe olan tahmini uzaklık ($h(n)$) hesaplarını birleştirir. Bu iki hesaplara birlikte $f(n) = g(n) + h(n)$ şeklinde en düşük maliyetli yolu bulmaya çalışır.

Özellikler:

- En kısa yolu garanti eder.
- Heuristik fonksiyonlarla daha hızlı çözüme ulaşır.
- Zaman karmaşıklığı $O(b^d)$

Kullanılan Heuristik Fonksiyonlar:

- Manhattan Distance: Taşların hedef pozisyona olan yatay ve dikey mesafelerinin toplamı.
- Euclidean Distance: Taşların hedef pozisyona olan doğrudan (kuvvetli) mesafesi.

3. Problem Özelleştirmesi ve Test Senaryoları

Aşağıda 3 adet başlangıç durumu ve sonuçların ekran çıktısı verilmiştir.

1. senaryo: 1 3 4
8 2 0
7 6 5

1 numaralı görseller bu durumun çalıştırılmasının sonucudur.

2. senaryo: 8 1 3
0 2 4
7 6 5

2 numaralı görseller bu durumun çalıştırılmasının sonucudur.

3. senaryo: 1 2 3
0 8 4
7 6 5

3 numaralı görseller bu durumun çalıştırılmasının sonucudur.

4. Algoritmaların Performans Karşılaştırması

1. Senaryo:

BFS algoritması ile çözüldüğünde : Üç adımda çözüldü.Sonuca ulaşma süresi 0.0009 saniye.Açılan düğüm sayısı 10 ve derinlik 3'tür.

DFS algoritması ile çözüldüğünde : Üç adımda çözüldü.Sonuca ulaşma süresi 0.001 saniye.Açılan düğüm sayısı 3 ve derinlik 3'tür.

A* algoritması ile çözüldüğünde : Üç adımda çözüldü.Sonuca ulaşma süresi 0.01 saniyedir.Açılan düğüm sayısı 3 ve derinlik 3'tür.

Sonuç: BFS, genişlik öncelikli arama yaptığı için her seviyedeki tüm düğümleri sırayla açar. Bu senaryoda derinlik 3 olmasına rağmen 10 düğüm açılmış ve 0.0009 saniye gibi oldukça kısa bir sürede çözüm sağlanmıştır. BFS her zaman optimal yolu bulur, ancak bellek maliyeti açılan düğüm sayısına bağlı olarak artar. Bu örnekteki düşük maliyet, çözümün erken seviyelerde bulunması sayesinde.

DFS, derinlik öncelikli arama yaptığı için doğrudan derinlemesine gider ve bu sayede sadece 3 düğüm açarak çözümü bulmuştur. Bu, bellek açısından oldukça verimli bir sonuçtur. Ayrıca 0.001 saniye ile son derece hızlı çözüm bulmuştur. Bu örnekte DFS, optimal yolu şanslı bir arama sırası ile bulmuştur, ancak her zaman optimal çözüm garantisi yoktur.

A* algoritması, heuristic fonksiyon sayesinde en optimal çözüm yolunu bulmak üzere tasarlanmıştır. Bu senaryoda yalnızca 3 düğüm açmış ve çözümü bulmuştur. Ancak 0.01 saniye ile BFS ve DFS'ten daha yavaş kalmıştır. Bu fark, A*'ın her adımda heuristic hesaplaması yapmasından kaynaklanır. Ancak doğru yol garantisi sağlar ve daha derin seviyelerde avantajı daha belirgin hale gelir.

2. Senaryo:

BFS algoritması ile çözüldüğünde : Üç adımda çözüldü.Sonuca ulaşma süresi 0.01 saniyedir.Açılan düğüm sayısı 10 ve derinlik 3 'tür.

DFS algoritması ile çözüldüğünde : Üç adımda çözüldü.Sonuca ulaşma süresi 0.001 saniyedir.Açılan düğüm sayısı 3 ve derinlik 3'tür.

A* algoritması ile çözüldüğünde : Üç adımda çözüldü.Sonuca ulaşma süresi 0.001 saniyedir.Açılan düğüm sayısı 3 ve derinlik 3'tür.

Sonuç: BFS, genişlik öncelikli arama yaptığı için her seviyedeki tüm düğümleri sırayla açar. Bu yüzden toplamda 10 düğüm açılmıştır. Problem küçük bir derinlikte çözümlenmesine rağmen, geniş kapsamlı arama yaptığı için daha fazla düğüm işlenmiştir. Bu da bellek maliyetini artıran bir faktördür. Sonuca 0.01 saniye gibi kısa bir sürede ulaşmıştır ancak bu, A* ve DFS'e kıyasla daha yavaştır. BFS, her zaman en kısa yolu bulur ama hafızada çok fazla düğüm tuttuğu için maliyetlidir.

DFS, derinlik öncelikli arama yaptığı için doğrudan bir yol üzerinde derinleşir ve yalnızca 3 düğüm açarak çözümü bulur. Bu, bellek açısından oldukça verimli bir çözüm sağlar. Ayrıca, sonuca ulaşma süresi 0.001 saniye ile en hızlı sonuç veren algoritma olmuştur. Ancak DFS, her zaman optimal çözümü garanti etmez; şanslı bir arama sırası olmuş diyebiliriz. Daha karmaşık yapılarda yanlış dallara sapması durumunda, süre ve bellek kullanımı ciddi şekilde artabilir.

A* algoritması, hem maliyeti (yolun uzunluğu) hem de tahmini uzaklığı (heuristic) hesaba kattığı için en optimal çözümü bulur. Yalnızca 3 düğüm açarak çözümü bulmuş ve 0.0 saniye ile en hızlı sonucu vermiştir. Burada, iyi tanımlanmış bir heuristic fonksiyonunun etkisi çok net bir şekilde görülmektedir. Ayrıca BFS gibi gereksiz düğüm açmadığı için bellek kullanımını da minimumda tutmuştur.

3. Senaryo:

BFS algoritması ile çözüldüğünde : Bir adımda çözüldü. Sonuca ulaşma süresi 0.003 saniyedir. Açılan düğüm sayısı 4 ve derinlik 1'dir.

DFS algoritması ile çözüldüğünde : Bir adımda çözüldü. Sonuca ulaşma süresi 68.99 saniyedir. Açılan düğüm sayısı 181439 ve derinlik 1'tür.

A* algoritması ile çözüldüğünde : Bir adımda çözüldü. Sonuca ulaşma süresi 0.001 saniyedir. Açılan düğüm sayısı 1 ve derinlik 1'dir.

Sonuç: BFS, genişlik öncelikli arama yaptığı için her seviyedeki tüm düğümleri sırayla açar. Bu senaryoda çözüm ilk seviyede bulunduğu için yalnızca 4 düğüm açılmış ve 0.003 saniye gibi oldukça kısa bir sürede çözüm sağlanmıştır. Bu, BFS'nin küçük problemler için hem hızlı hem de bellek dostu olabileceğini gösterir. Ancak çözüm daha derinlerde olsaydı, bellek kullanımı katlanarak artacaktı. BFS, her zaman en kısa yolu garanti eder.

DFS, derinlik öncelikli arama yaptığı için tek bir yol üzerinde derinlemesine ilerler. Bu örnekte çözüm aslında ilk seviyede olmasına rağmen DFS, 181439 düğüm açarak gereksiz bir arama gerçekleştirmiştir. Bu, DFS'in kontrolsüz derinleşmesinden ve arama sırasındaki verimsizlikten kaynaklanır. Ayrıca 68.99 saniye gibi inanılmaz yüksek bir süre harcamıştır. Bu, DFS'in kötü bir arama stratejisi izlediğinin göstergesidir. Bellek kullanımı da son derece yüksektir ve büyük problemler için bu kadar büyük bir maliyet kabul edilemez.

A* algoritması, heuristic fonksiyon sayesinde çözüm yolunu akılcıca tahmin eder. Bu senaryoda sadece 1 düğüm açarak en hızlı çözümü (0.001 saniye) bulmuştur. Bu durum, iyi tanımlanmış bir heuristic fonksiyonunun ne kadar büyük bir fark yaratabileceğini gösterir. Hem bellek hem de zaman açısından en verimli çözümü sağlamıştır.

5. Sonular ve Deęerlendirme

Proje kapsamında BFS, DFS ve A* algoritmaları başarıyla uygulanmış ve 8-Puzzle probleminin özümü elde edilmiştir. Yapılan testler ve analizler doęrultusunda, A* algoritmasının Manhattan Mesafesi ile en kısa sürede özüm sağladığı gözlemlenmiştir. BFS algoritması doęru sonuç verse de bellek kullanımı yüksek seviyelerde kalmıştır. DFS algoritması ise bellek açısından avantaj sağlasa da bazı durumlarda optimal özümü garanti edememektedir.

6. Kaynaka

<https://github.com/MarkAMoussa/8-puzzle-A--BFS-DFS-Python/tree/master>

<https://www.geeksforgeeks.org/a-search-algorithm/>

<https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/>

<https://www.redblobgames.com/pathfinding/a-star/introduction.html>

<https://chatgpt.com/>

7. Ekran Çıktıları

1. başlangıç durumu - BFS Algoritması

```
***** welcome to 8-puzzle game *****
please enter initial state ::
File 1: 1
File 2: 3
File 3: 4
File 4: 8
File 5: 2
File 6: 0
File 7: 7
File 8: 6
File 9: 5
Entered state:
1 3 4
8 2 0
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
1
*****
you chose BFS algorithm
solving the puzzle please wait....
path taken to reach goal >>

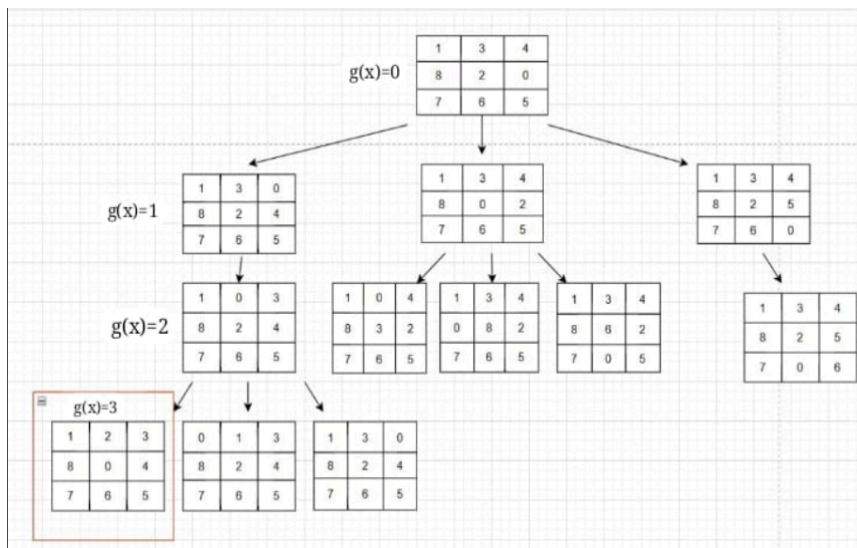
Step 0:
[1 3 4]
[8 2 0]
[7 6 5]

Step 1:
[1 3 0]
[8 2 4]
[7 6 5]

Step 2:
[1 0 3]
[8 2 4]
[7 6 5]

Step 3:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['UP', 'LEFT', 'DOWN']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 3
nodes expanded >> 10
depth is 3
running time of BFS is >> 0.0009999275207519531 secs
*****
please enter initial state ::
File 1: 
```



1. başlangıç durumu - DFS Algoritması

```
***** welcome to 8-puzzle game *****
please enter initial state ::
Tile 1: 1
Tile 2: 3
Tile 3: 4
Tile 4: 8
Tile 5: 2
Tile 6: 0
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
1 3 4
8 2 0
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
2
*****
you chose DFS algorithm
solving the puzzle please wait....
path taken to reach goal >>

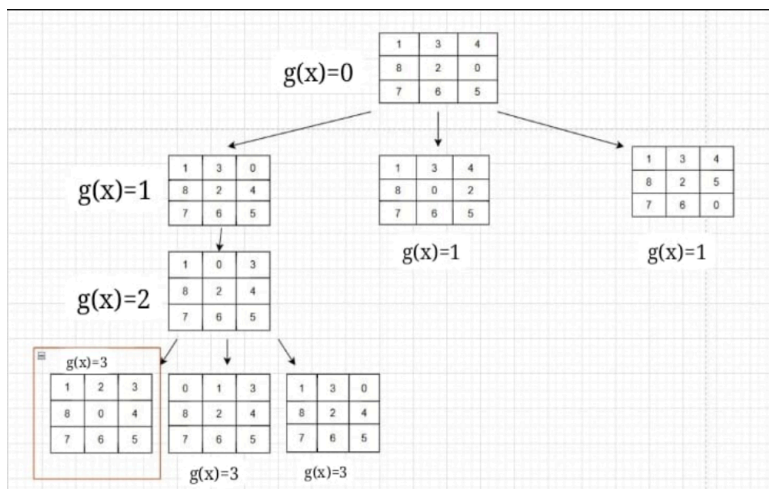
Step 0:
[1 3 4]
[8 2 0]
[7 6 5]

Step 1:
[1 3 0]
[8 2 4]
[7 6 5]

Step 2:
[1 0 3]
[8 2 4]
[7 6 5]

Step 3:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['UP', 'LEFT', 'DOWN']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 3
nodes expanded >> 3
depth is 3
running time of DFS is >> 0.0 secs
*****
please enter initial state ::
Tile 1: 
```



1. başlangıç durumu - A* Algoritması

```
***** welcome to 8-puzzle game *****
please enter initial state ::
Tile 1: 1
Tile 2: 3
Tile 3: 4
Tile 4: 8
Tile 5: 2
Tile 6: 0
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
1 3 4
8 2 0
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
3
please choose the heuristic function to be used ::
1- manhattan
2- euclidean
1
*****
you chose A* algorithm
solving the puzzle please wait....
path taken to reach goal >>

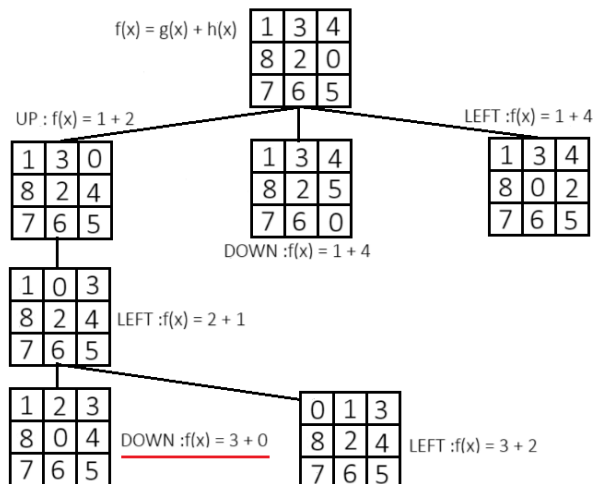
Step 0:
[1 3 4]
[8 2 0]
[7 6 5]

Step 1:
[1 3 0]
[8 2 4]
[7 6 5]

Step 2:
[1 0 3]
[8 2 4]
[7 6 5]

Step 3:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['UP', 'LEFT', 'DOWN']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 3
nodes expanded >> 3
depth is 3
running time of A* is >> 0.8 secs
*****
please enter initial state ::
Tile 1: 1 for history. Search history with c-f/c-l
```



2. başlangıç durumu - BFS Algoritması

```
EightPuzzle.ipynb | Python 3 (pykernel)
***** welcome to 8-puzzle game *****
*****
please enter initial state ::
Tile 1: 8
Tile 2: 1
Tile 3: 3
Tile 4: 0
Tile 5: 2
Tile 6: 4
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
8 1 3
0 2 4
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
1
*****
you chose BFS algorithm
solving the puzzle please wait....
path taken to reach goal >>

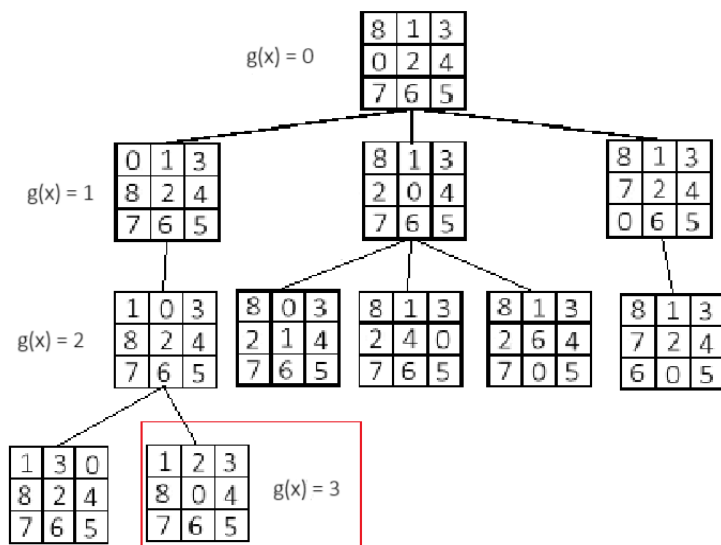
Step 0:
[8 1 3]
[0 2 4]
[7 6 5]

Step 1:
[0 1 3]
[8 2 4]
[7 6 5]

Step 2:
[1 0 3]
[8 2 4]
[7 6 5]

Step 3:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['UP', 'RIGHT', 'DOWN']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 3
nodes expanded >> 10
depth is 3
running time of BFS is >> 0.011791706085205078 secs
```



2. başlangıç durumu - DFS Algoritması

```
EightPuzzle.ipynb
+ + + + +
please enter initial state ::
Tile 1: 8
Tile 2: 1
Tile 3: 3
Tile 4: 0
Tile 5: 2
Tile 6: 4
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
8 1 3
0 2 4
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
2
*****
you chose DFS algorithm
solving the puzzle please wait....
path taken to reach goal >>

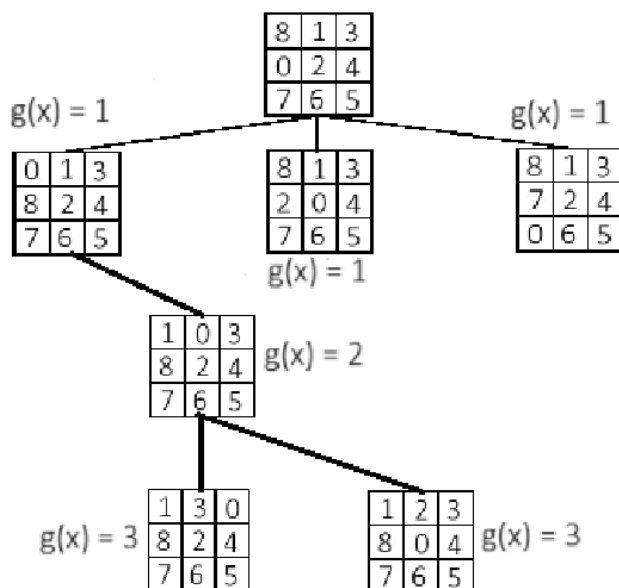
Step 0:
[8 1 3]
[0 2 4]
[7 6 5]

Step 1:
[0 1 3]
[8 2 4]
[7 6 5]

Step 2:
[1 0 3]
[8 2 4]
[7 6 5]

Step 3:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['UP', 'RIGHT', 'DOWN']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 3
nodes expanded >> 3
depth is 3
running time of DFS is >> 0.0013322830200195312 secs
```



2. başlangıç durumu - A* Algoritması

```

***** welcome to 8-puzzle game *****
please enter initial state ::
Tile 1: 8
Tile 2: 1
Tile 3: 3
Tile 4: 0
Tile 5: 2
Tile 6: 4
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
8 1 3
0 2 4
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
3
please choose the heuristic function to be used ::
1- manhattan
2- euclidean
1
*****
you chose A* algorithm
solving the puzzle please wait....
path taken to reach goal >>

Step 0:
[8 1 3]
[0 2 4]
[7 6 5]

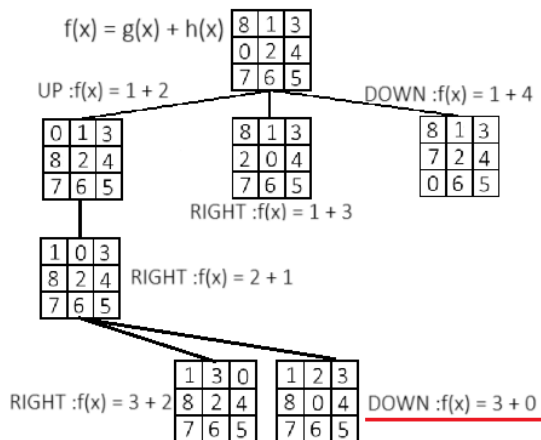
Step 1:
[0 1 3]
[8 2 4]
[7 6 5]

Step 2:
[1 0 3]
[8 2 4]
[7 6 5]

Step 3:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['UP', 'RIGHT', 'DOWN']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 3
nodes expanded >> 3
depth is 3
running time of A* is >> 0.0 secs
*****
please enter initial state ::

```



3. başlangıç durumu - BFS Algoritması

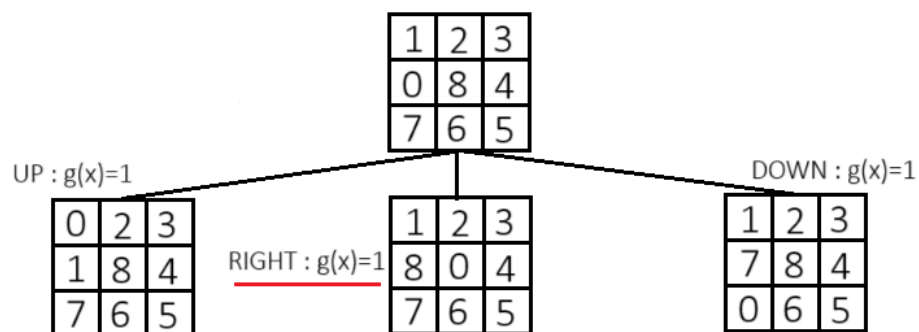
```
EightPuzzle.ipynb x +
Notebook Python 3 (ipykernel)

***** welcome to 8-puzzle game *****
*****
please enter initial state ::
Tile 1: 1
Tile 2: 2
Tile 3: 3
Tile 4: 0
Tile 5: 8
Tile 6: 4
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
1 2 3
0 8 4
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
1
*****
you chose BFS algorithm
solving the puzzle please wait....
path taken to reach goal >>

Step 0:
[1 2 3]
[0 8 4]
[7 6 5]

Step 1:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['RIGHT']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 1
nodes expanded >> 4
depth is 1
running time of BFS is >> 0.0034570693969726562 secs
```



3. başlangıç durumu - DFS Algoritması

```

EightPuzzle.ipynb
+ + + + +
Notebook Python 3 (ipykernel)

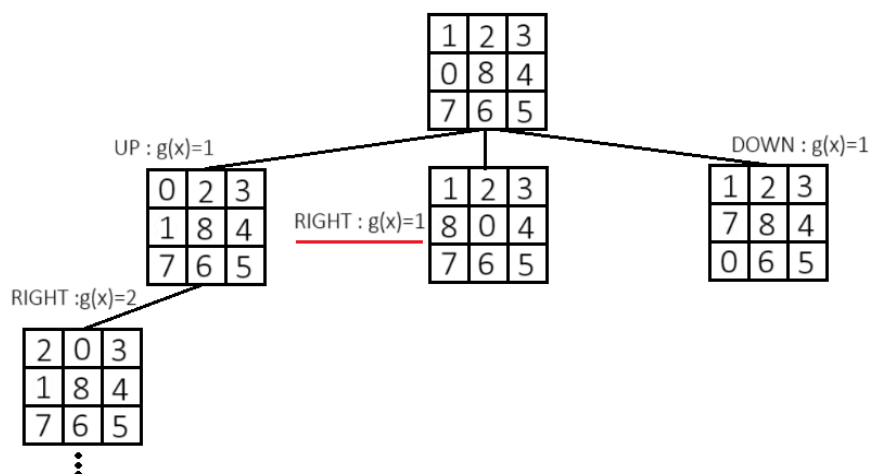
*****
please enter initial state ::
Tile 1: 1
Tile 2: 2
Tile 3: 3
Tile 4: 0
Tile 5: 8
Tile 6: 4
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
1 2 3
0 8 4
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
2
*****
you chose DFS algorithm
solving the puzzle please wait...
path taken to reach goal >>

Step 0:
[1 2 3]
[0 8 4]
[7 6 5]

Step 1:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['RIGHT']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 1
nodes expanded >> 181439
depth is 1
running time of DFS is >> 68.993323802948 secs

```



3. başlangıç durumu - A* Algoritması

```
EightPuzzle.ipynb
+ - Code Python 3 (ipykernel)

*****
please enter initial state ::
Tile 1: 1
Tile 2: 2
Tile 3: 3
Tile 4: 0
Tile 5: 8
Tile 6: 4
Tile 7: 7
Tile 8: 6
Tile 9: 5
Entered state:
1 2 3
0 8 4
7 6 5
*****
select the algorithm to solve the puzzle ::
1- BFS
2- DFS
3- A*
4- exit
3
please choose the heuristic function to be used ::
1- manhattan
2- euclidean
1
*****
you chose A* algorithm
solving the puzzle please wait....
path taken to reach goal >>

Step 0:
[1 2 3]
[0 8 4]
[7 6 5]

Step 1:
[1 2 3]
[8 0 4]
[7 6 5]

direction taken to reach goal >> ['RIGHT']
goal reached >>
[1 2 3]
[8 0 4]
[7 6 5]
total cost to reach goal is 1
nodes expanded >> 1
depth is 1
running time of A* is >> 0.0010275840759277344 secs
```

